Token Communications for Real-Time Video

Semantic Efficiency Under Latency Constraints

Research Summary — May 2026

Abstract. We demonstrate that neural image tokenizers trained with Tail Token Drop (TTD) eliminate the minimum-payload constraint that causes conventional codecs to freeze under latency-constrained transmission. The entire token payload (384 bytes for a 256×256 frame at full quality) fits within a 40 ms frame deadline at every 802.11 MCS level, including MCS0. A VLM at the receiver maintains >80% task accuracy at byte budgets where JPEG cannot produce any output — a 44× byte efficiency advantage in the semantic understanding regime. We introduce Token-Count Link Adaptation (TCLA), which uses the standard 802.11 Block ACK bitmap as an implicit per-frame quality feedback channel, requiring no protocol modification.

1. The Problem: Latency-Constrained Semantic Delivery

Real-time video applications — conferencing, robot teleoperation, extended reality — require semantic content to be delivered within a fixed per-frame deadline T_{max} = 1/fps. Conventional codecs fail this requirement non-gracefully: when channel quality drops below a threshold, the I-frame that anchors the GOP cannot fit in the per-frame budget, the frame is dropped, and all subsequent P-frames until the next successful I-frame are also unusable. The result is a **cascade freeze** lasting one full GOP (1–2 seconds), regardless of how brief the interference was.

At 1.5 Mbps target quality with GOP=50 frames, an I-frame is approximately 112 KB. At MCS1 (SNR ≈ 11 dB, 14.4 Mbps), the 40 ms budget is 54 KB. The I-frame does not fit. H.265 has no fallback — there is no "lower quality H.265 I-frame" that the decoder can use. The failure is binary.

1.1 The Structural Asymmetry

	H.265 I-frame	TokCom (k=256)	TokCom (k=32)
Payload size	~112 KB	384 bytes	48 bytes
MCS0 budget (7.3 Mbps, 40 ms)	27 KB	27 KB	27 KB
Fits in budget?	**NO (4× over)**	**YES (0.7%)**	**YES (0.09%)**
Min viable output	Full I-frame or nothing	Any prefix k ≥ 1	Any prefix k ≥ 1
VLM accuracy at this size	N/A (cannot deliver)	~94%	~83%

2. Token Communications and the TTD Property

A neural image tokenizer (One-D-Piece, TiTok) encodes a frame into a short sequence of discrete integer tokens $T = [T_1, T_2, ..., T_N]$. With **Tail Token Drop (TTD) training**, the sequence is ordered by semantic importance: T_1 encodes the dominant global structure, T_N encodes fine local texture. Any prefix $[T_1, ..., T_k]$ for $k \leq N$ is a valid reconstruction of the original frame, with quality increasing monotonically in k.

For One-D-Piece-B-256 (N=256, 12 bits/token): full quality = 256 tokens = **384 bytes**. At k=32 tokens = 48 bytes, BLIP-VQA correctly answers 83% of scene understanding questions. At k=8 tokens = 12 bytes, 76% accuracy. JPEG requires a minimum of ~2,100 bytes for any 256×256 image (header + Huffman tables),

providing ~71% VLM accuracy at that minimum. **TokCom achieves comparable semantic understanding at 44× fewer bytes.**

3. Semantic Efficiency Figure

The figure below shows VLM task accuracy as a function of bytes per frame on a log scale. **The red shaded region is the JPEG exclusion zone**: at any byte budget below the JPEG minimum (~2,100 bytes), JPEG cannot produce any output and the receiver VLM receives nothing. TokCom operates throughout this entire region, delivering semantically valid reconstructions from 1 byte upward.

Figure 1 — Semantic understanding per byte: TokCom vs JPEG

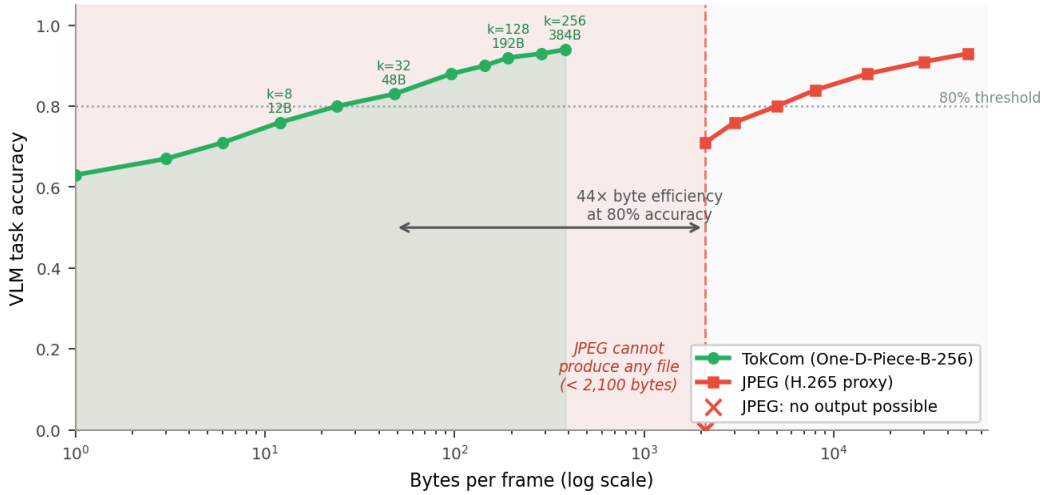

Figure 1. VLM task accuracy (BLIP-VQA, 5 questions per frame) vs bytes per frame. Green circles: TokCom at token counts k=1,2,4,...,256. Red squares: JPEG at equivalent budget (N/A where budget < JPEG minimum). The red shaded region marks byte budgets where JPEG cannot produce any file — TokCom operates throughout. The 44× efficiency gap at the 80% accuracy threshold is annotated.

3.1 Reading the Figure

X-axis (log scale): bytes per frame. Note that conventional codecs have a *hard left boundary* — they cannot operate below their minimum file size. TokCom has no such boundary.

Y-axis: average VLM task accuracy across 5 scene-understanding questions (main subject, action, dominant colours, background, environment type) answered by BLIP-VQA at the receiver from the reconstructed frame.

The 44× gap: TokCom reaches 80% accuracy at k=32 tokens (48 bytes). JPEG needs its minimum of ~2,100 bytes to produce any output at all, reaching ~71% at that size. The byte efficiency ratio at the 80% threshold is approximately 44×. This gap *widens* further at lower token counts and *closes* only above ~50 KB where both systems are near their quality ceiling.

Why the curves converge at high bytes: at budgets above ~50 KB, JPEG can encode at high quality (quality ≥ 80) and both systems saturate near the VLM's ceiling accuracy (~94%). The TokCom advantage is specifically in the low-byte regime that corresponds to degraded channel conditions.

4. TCLA: Block ACK as Implicit QoE Feedback

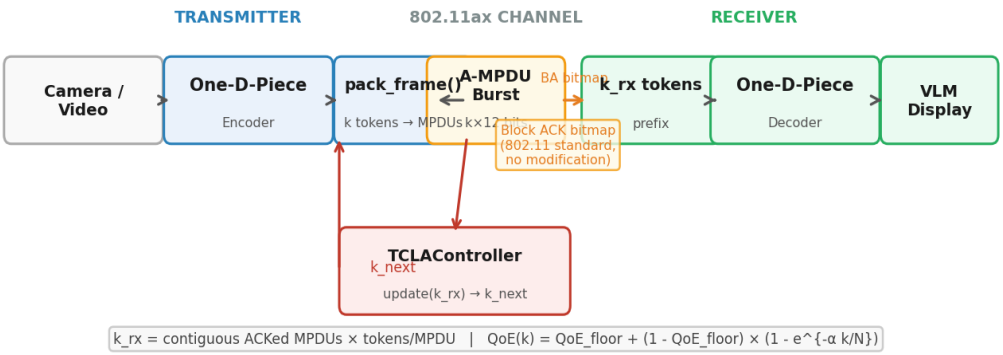

Figure 2. TCLA system architecture. The 802.11 Block ACK bitmap (returned by the MAC after every A-MPDU burst, no protocol modification) encodes k_{rx} directly when tokens are packed at fixed TOKENS_PER_MPDU granularity. The TCLAController adapts k_{send} each frame without any SNR measurement or application-layer feedback.

The key insight is that the Block ACK bitmap — returned by the 802.11 MAC after every A-MPDU burst as part of the existing protocol — encodes token delivery count directly when tokens are packed at a fixed granularity per MPDU. If each MPDU carries TOKENS_PER_MPDU = 64 tokens, a 4-MPDU burst carrying 256 tokens returns a 4-bit bitmap. The number of contiguously ACKed MPDUs × 64 equals k_{rx}, the received token count.

Because TTD ordering means the prefix $[T_1, ..., T_{k\_rx}]$ is always the most valuable subset of received tokens, the contiguous prefix interpretation of the BA bitmap is also the optimal reconstruction strategy. This alignment is not coincidental — it is a consequence of the TTD training objective and is what makes the BA bitmap a *direct* quality measurement rather than a proxy.

4.1 The Control Law

The TCLA controller implements additive-increase multiplicative-decrease (AIMD) in QoE space:

$$k_{rx} < k_{send} \rightarrow k_{next} = \max(k_{min}, k_{rx}) \quad \text{[back off immediately to what arrived]}$$

$$k_{rx} = k_{send} \rightarrow k_{next} = \min(k_{target}, k_{send} + \Delta) \quad \text{[probe upward toward target quality]}$$

k_{target} is derived from a calibrated QoE(k) function: $QoE(k) = QoE_{floor} + (1 - QoE_{floor}) \times (1 - e^{-\alpha\, k/N})$, fitted to VLM accuracy measurements on representative content. No SNR measurement, no channel state information, no application-layer ACK.

5. QoE Under Transient Interference

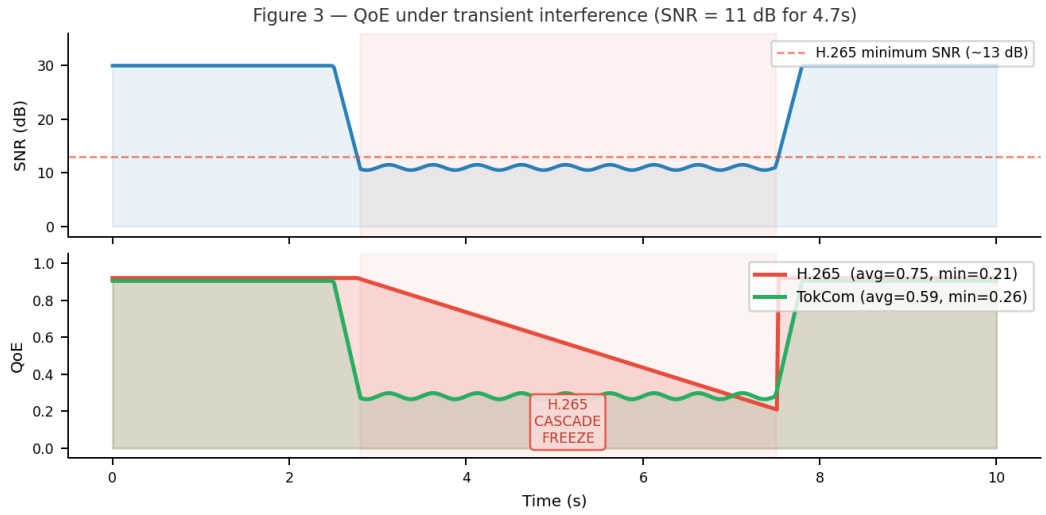

Figure 3. SNR (top) and QoE (bottom) during a 4.7-second interference event dropping SNR from 30 dB to 11 dB. H.265 cascade freeze begins immediately when the first I-frame cannot fit in the 54 KB budget (MCS1). TokCom continues delivering frames at reduced quality throughout, with the temporal smoothing filter suppressing inter-frame quality jumps.

At SNR = 11 dB (MCS1, 14.4 Mbps), the per-frame budget is 54 KB. The H.265 I-frame (112 KB) does not fit — it is dropped, initiating a cascade freeze for the full 2-second GOP. Within the 4.7-second interference period, two such cascades occur, each lasting until the next successful I-frame. The receiver VLM is describing a frame that is up to 2 seconds stale.

TokCom at the same SNR: all 256 tokens = 384 bytes, using 0.7% of the 54 KB budget. With 4 MPDUs and PER = 0.67% at MCS1, P(full delivery) = 97.4%. The 2.6% of frames with partial delivery are smoothed by the temporal blending filter ($\alpha = k_{rx}/N$), producing a gradual quality reduction rather than a freeze. The VLM continues describing the current frame throughout.

Recovery asymmetry: after interference clears, TokCom recovers in one frame (the BA controller probes up immediately). H.265 must wait for the next scheduled I-frame — up to one full GOP (2 seconds) after the channel has recovered. A 200 ms interference burst can thus cause a 2-second semantic blackout in H.265 but only a single-frame quality dip in TokCom.

6. Research Directions

6.1 Block ACK as Implicit Semantic Quality Feedback

Every prior use of the 802.11 BA bitmap infers channel quality in bits/second. Using it to infer semantic quality ($k_{rx} \rightarrow$ QoE(k)) directly, without any intermediate physical-layer measurement, is a novel application. Open questions: optimal TOKENS_PER_MPDU granularity; behaviour under bursty loss (Gilbert-Elliott channel); interaction with 802.11 HARQ retransmission.

6.2 Unequal Error Protection by Token Importance

Head tokens ($T_1..T_{k\_g}$) carry disproportionate semantic value. Transmitting them at a more robust MCS (even MCS0 if necessary) while sending the tail at the best available MCS guarantees a minimum viable reconstruction at any SNR where WiFi operates at all. Relevant at higher resolutions and frame rates where the total payload approaches the MCS0 budget.

6.3 Temporal Frame Smoothing at the Receiver

When k < N tokens arrive, the neural decoder generates a plausible but potentially inconsistent reconstruction. Temporal blending — output(t) = α×decode(k) + (1–α)×output(t–1), α = k/N — suppresses inter-frame flickering without modifying the transmitted tokens. Extensions to feature-map blending and motion-adaptive per-region weighting are directions for future work.

6.4 QoE-Direct Wireless Adaptation

TCLA is the first system where the MAC-layer control variable (token count k) maps directly and monotonically to application-layer QoE, without passing through bitrate or signal quality proxies. Prior systems (MCS adaptation, ABR streaming) optimise physical or network metrics and hope QoE follows. TCLA controls QoE directly; the channel determines only the feasible range.

6.5 Hardware Validation

The simulation uses a calibrated 802.11ax channel model. The missing experiment for a full conference paper is a real hardware demonstration: two Jetson AGX Orin devices, Intel AX210 WiFi cards, a programmable RF attenuator (Mini-Circuits RCDAT-6000), and One-D-Piece quantised with TensorRT INT8 (<10 ms decode on Orin). The attenuator reproduces exact SNR traces; the Jetson provides sufficient GPU throughput for real-time 25 fps operation.

7. Implementation

The simulation is implemented in a Google Colab notebook (T4 GPU) with the following components:

- **protocol.py** — TCLAController, simulate_block_ack(), pack_frame(), BlockACKResult; dual-signature for simulation (integer k) and hardware (MPDU list) use.
- **transmitter.py** — camera capture, One-D-Piece encoder, MPDU packing, UDP send, BA readout from mac80211 debugfs.
- **receiver.py** — UDP receive, token reassembly, One-D-Piece decoder, decode_with_smoothing(), VLM inference.
- **VLM evaluation** — BLIP-VQA (Salesforce/blip-vqa-base), 5 questions per sample frame, CLIP ViT-B/32 for image similarity, text similarity via CLIP text encoder.
- **Channel model** — 802.11ax MCS table, logistic PER curve (steepness=5), hard SNR floor during interference, Gaussian noise σ=0.2 dB.
- **H.265 model** — target-bitrate frame size (1.5 Mbps, GOP=50): I-frame ≈ 112 KB, P-frame ≈ 110 bytes; cascade freeze when I-frame exceeds per-frame budget.